

```
[83]: # improve Logistic Regression properly
      # Start by creating a balanced Logistic Regression pipeline:
```

```
[84]: balanced_logistic_model = Pipeline([
      ("preprocessor", preprocessor),
      ("classifier", LogisticRegression(
          max_iter=1000,
          class_weight="balanced"
      ))
  ])
])
```

```
[85]: balanced_logistic_model.fit(X_train, y_train)

y_pred_balanced = balanced_logistic_model.predict(X_test)
y_proba_balanced = balanced_logistic_model.predict_proba(X_test)[: , 1]
```

```
[86]: print("Accuracy:", accuracy_score(y_test, y_pred_balanced))

print("\nClassification Report:")
print(classification_report(y_test, y_pred_balanced))

print("ROC-AUC:", roc_auc_score(y_test, y_proba_balanced))
```

Accuracy: 0.7381121362668559

Classification Report:

	precision	recall	f1-score	support
No	0.90	0.72	0.80	1035
Yes	0.50	0.78	0.61	374
accuracy			0.74	1409
macro avg	0.70	0.75	0.71	1409
weighted avg	0.80	0.74	0.75	1409

ROC-AUC: 0.8416388953473353

```
[87]: for threshold in [0.5, 0.4, 0.3, 0.2]:
      y_pred_threshold = np.where(
          y_proba_balanced >= threshold,
          "Yes",
          "No"
      )

      print(f"Threshold: {threshold}")
      print("Precision:", precision_score(
          y_test, y_pred_threshold, pos_label="Yes"
      ))
      print("Recall:", recall_score(
          y_test, y_pred_threshold, pos_label="Yes"
      ))
      print("F1:", f1_score(
          y_test, y_pred_threshold, pos_label="Yes"
      ))
      print()
```

Threshold: 0.5
Precision: 0.504302925989673
Recall: 0.7834224598930482
F1: 0.6136125654450262

Threshold: 0.4
Precision: 0.46618705035971225
Recall: 0.8663101604278075
F1: 0.6061739943872778

Threshold: 0.3
Precision: 0.42998760842627015
Recall: 0.9278074866310161
F1: 0.5876375952582558

Threshold: 0.2
Precision: 0.38876889848812096
Recall: 0.9625668449197861
F1: 0.5538461538461539

```
[89]: # Hyperparameter tuning
```

```
[90]: from sklearn.model_selection import GridSearchCV

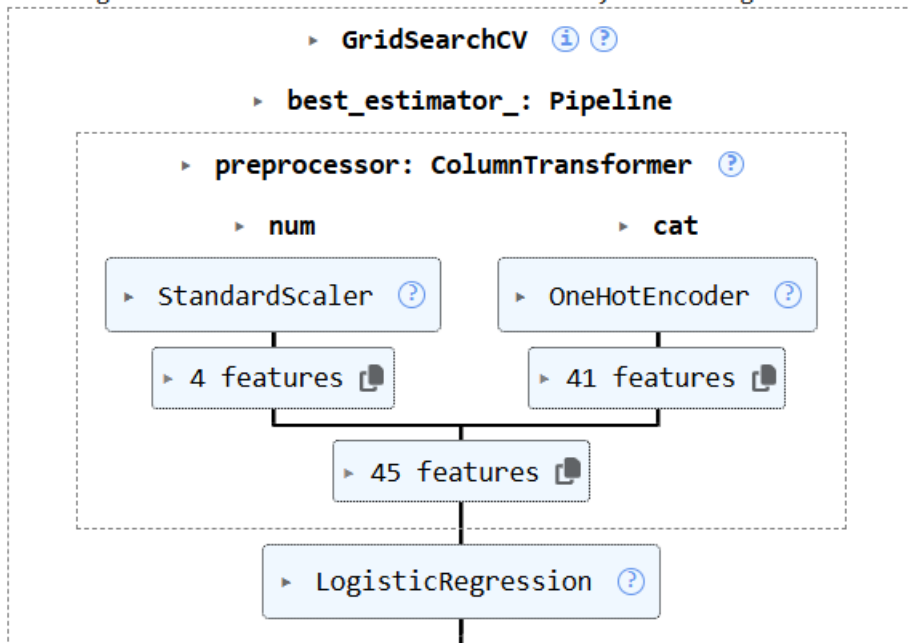
param_grid = {
    "classifier__C": [0.01, 0.1, 1, 10, 100],
    "classifier__class_weight": [None, "balanced"],
    "classifier__solver": ["liblinear", "lbfgs"]
}
```

```
[91]: grid_search = GridSearchCV(
    estimator=logistic_model,
    param_grid=param_grid,
    scoring="f1_macro",
    cv=5,
    n_jobs=-1,
    verbose=1
)
```

```
[92]: grid_search.fit(X_train, y_train)
```

Fitting 5 folds for each of 20 candidates, totalling 100 fits

92]:



93]:

```
print("Best parameters:")
print(grid_search.best_params_)

print("\nBest CV score:")
print(grid_search.best_score_)
```

Best parameters:

```
{'classifier_C': 100, 'classifier_class_weight': None, 'classifier_solver': 'liblinear'}
```

Best CV score:

```
0.7352967057095116
```

[94]: *# Evaluate the tuned model*

[95]: `best_model = grid_search.best_estimator_`

```
y_pred_tuned = best_model.predict(X_test)
y_proba_tuned = best_model.predict_proba(X_test)[:, 1]
```

[96]: `print("Accuracy:", accuracy_score(y_test, y_pred_tuned))`

```
print("\nClassification Report:")
print(classification_report(y_test, y_pred_tuned))
```

```
print("ROC-AUC:", roc_auc_score(y_test, y_proba_tuned))
```

Accuracy: 0.801277501774308

Classification Report:

	precision	recall	f1-score	support
No	0.85	0.89	0.87	1035
Yes	0.65	0.55	0.60	374
accuracy			0.80	1409
macro avg	0.75	0.72	0.73	1409
weighted avg	0.79	0.80	0.80	1409

ROC-AUC: 0.8408483815133432